

CA Agent

An AI-Powered Chartered Accountant Assistant

Final Year Project — Current State, Proposed Extensions, and Academic Justification

B.E./B.Tech Computer Engineering · Academic Year 2024–25

Hardware:	Intel Core i5 12th Gen, 16 GB RAM, 4 GB VRAM
Stack:	Python (FastAPI) · React + TypeScript · Gemini Vision · sentence-transformers
Domain:	Indian taxation, GST, ICAI study material, bank statement analysis

1. Abstract

CA Agent is a domain-specific AI assistant designed to help Chartered Accountants and their clients with tax planning, GST compliance, and financial document analysis. The system combines Retrieval-Augmented Generation (RAG) over ICAI-aligned embeddings with Gemini Vision-based document extraction, delivered through a conversational chat interface.

This document describes the current working prototype, the technical and academic limitations it carries, and a set of targeted extensions that transform it from a prototype into a research-grade final year project. Each extension is justified on three axes: practical value to the domain, technical novelty relative to the state of the art, and measurable academic contribution suitable for a thesis or project report.

2. Problem Statement

India has over 3.5 lakh practicing Chartered Accountants serving millions of businesses and individuals. Their clients routinely struggle with:

- Understanding which ITR form applies to their income profile.
- Tracking GST filing deadlines across GSTR-1, GSTR-3B, and annual returns.
- Interpreting bank statements and estimating advance tax liability.
- Navigating the 1,000+ sections of the Income Tax Act and GST legislation.

Existing tools (ClearTax, Quicken, TaxBuddy) are form-filling utilities. They do not provide conversational explanations, cannot reason over uploaded financial documents, and offer no retrieval from authoritative ICAI study material. CA Agent addresses this gap by combining document intelligence, RAG-based legal retrieval, and a conversational interface.

3. Current State of the Project

The prototype is a fully functional, end-to-end system. This section documents what is already built, so the examiner can distinguish the baseline from the proposed extensions.

3.1. Architecture Overview

What is already built and working

- **Document extraction:** Users upload bank statements (PDF, PNG, JPG, WEBP). `file_handler.py` sends them to Gemini Vision with a strict JSON prompt that extracts `person_entity`, `statement_period`, `opening_balance`, `total_credits`, `estimated_annual_income`, and tax breakdown fields.
- **RAG pipeline:** `search.py` loads three precomputed embedding corpora (`embeddings_foundation.joblib`, `embeddings_Intermediate.joblib`, `embeddings_Final.joblib`) corresponding to ICAI Foundation, Intermediate, and Final study levels. User queries are embedded via `bge-m3` through a local Ollama server, cosine similarity selects the top candidates, and a CrossEncoder (`cross-encoder/ms-marco-MiniLM-L-6-v2`) reranks them.
- **Optional HyDE:** `hyde.py` generates a hypothetical document for the query when `ENABLE_HYDE=true`, blending the hypothetical embedding with the raw query embedding before retrieval.
- **Conversational agent:** `ca_agent.py` combines retrieved chunks, uploaded document context, and chat history to generate answers via Gemini.
- **Frontend:** React + TypeScript with Tailwind CSS and shadcn/ui. Chat state is persisted in browser `localStorage`. Supports multi-conversation management, file upload, and retrieval preview rendering.
- **Backend:** FastAPI server (`api.py`) with `/upload` and `/ask` endpoints. Limits payload size, builds 3-turn chat memory, and returns structured `doc_data` for frontend persistence.

3.2. Technology Stack

Layer	Technology
Frontend	React 18, TypeScript, Vite, Tailwind CSS, shadcn/ui, react-router-dom, @tanstack/react-query, react-markdown, recharts
Backend	Python 3.11, FastAPI, Uvicorn, Pydantic, python-dotenv
ML / Retrieval	sentence-transformers, scikit-learn (cosine similarity), Ollama (bge-m3), CrossEncoder (ms-marco-MiniLM-L-6-v2)
LLM	Google Gemini 1.5 Pro (answers), Gemini Vision (document extraction), Gemini Flash (HyDE, optional)
Storage	Precomputed .joblib embeddings, browser localStorage

3.3. Data Flow

Upload flow: User uploads document → `api.py` saves to `temp/` → `file_handler.py` calls Gemini Vision → structured JSON returned as `doc_data` → stored in frontend conversation state.

Query flow: User submits question → frontend posts `{question, history, doc_data, plain_file_texts}` to `/ask` → `search.py` embeds query, retrieves and reranks chunks → `ca_agent.py` assembles prompt with document context and retrieved chunks → Gemini generates answer → response includes answer + retrieval previews.

3.4. Acknowledged Limitations of the Current System

Limitations (documented in project context)

1. **No backend session store.** Conversations live in browser `localStorage` only. Multiple devices, incognito mode, or a cleared browser loses all history.
2. **3-turn context window.** `api.py` builds chat memory from only the last 3 completed turns. Long tax planning sessions lose earlier context.
3. **Ollama dependency.** Embedding requires a separately running Ollama server. This prevents straightforward cloud deployment.
4. **Single-document isolation.** Each uploaded document is analysed independently. Aggregated financial analysis (e.g. quarterly income trends) is not possible.
5. **No retrieval quality measurement.** `metrics.py` exists but is gated by `ENABLE_METRICS` and has never been used systematically.
6. **No reliability signal.** There is no mechanism to distinguish answers grounded in ICAI-retrieved text from model-generated inference. In a legal-financial domain, this is a significant trust gap.
7. **Generic embeddings.** `bge-m3` was not trained on ICAI or Indian tax terminology. Domain-specific terms like “Section 44AD”, “GST-3B”, and “presumptive taxation” may not be well-represented.

Each proposed extension in Section 4 directly addresses one or more of these limitations.

4. Proposed Extensions

The following nine extensions are proposed. They are grouped by contribution type: infrastructure fixes, ML research contributions, and product features. Each subsection states the extension, shows a concrete usage example, justifies the academic contribution, and maps it to the limitation it resolves.

4.1. Extension 1: Long-Term Conversational Memory

Limitation addressed: #2 (3-turn context window).

What changes

The 3-turn history window in `api.py` is replaced with a rolling summary architecture. Older turns are compressed via a Gemini Flash summarisation call and stored as a structured `summary` field alongside the recent verbatim turns. The frontend stores this summary in `ChatContext` and passes it on every `/ask` request.

Concrete example: A user uploads an HDFC bank statement in message 1 (income Rs. 9.2L, TDS Rs. 48K). Fifteen messages later she asks: “*Based on what I uploaded, can I still make a 80C investment to reduce tax?*” Without rolling memory, the agent has no knowledge of the upload. With rolling memory, the summary carries the key financial facts throughout the session.

Academic justification

Conversational memory management is an active area in LLM application research (Zhu et al., 2023; MemGPT, Packer et al., 2023). Implementing and evaluating a domain-specific rolling summary strategy — where the compression prompt is tuned for financial context retention — constitutes a genuine design contribution. The thesis can measure memory fidelity: what percentage of key financial facts (income, deductions, document periods) survive summarisation across N turns.

4.2. Extension 2: Multi-Document Cross-Analysis

Limitation addressed: #4 (single-document isolation).

What changes

A new `merge_doc_data(docs)` function in `ca_agent.py` aggregates extracted fields across all uploaded documents in the active conversation. A new analytics dashboard (separate React route) visualises the aggregated data using recharts: monthly income vs. expense bar chart, TDS trend line, GST liability donut.

Concrete example: A self-employed consultant uploads ICICI statements for April, May, and June. The agent now computes: total Q1 income Rs. 2.7L, average monthly expense Rs. 38K, and flags an anomalous Rs. 80K outflow in May. It then estimates advance tax due for Q2 based on the extrapolated annual figure.

Academic justification

Multi-document financial reasoning is an understudied area in domain-specific RAG. The aggregation layer requires reconciling different statement periods, currency formats, and entity names from Gemini Vision’s JSON output — a non-trivial data engineering contribution. The dashboard provides visual evidence that the extraction pipeline is working correctly, which is independently evaluable.

4.3. Extension 3: Anomaly Detection on Extracted Transactions

Limitation addressed: Adds proactive intelligence beyond reactive Q&A.

What changes

An `IsolationForest` model (scikit-learn, CPU-only, no GPU required) is applied to the transaction amounts extracted by Gemini Vision. Flagged transactions are appended to the `summary_message` in the `/upload` response, with a short natural-language explanation of why each transaction is unusual.

Concrete example: Priya’s statement shows 11 monthly salary credits near Rs. 45K and one credit of Rs. 4.2L. The system flags: *“Unusual credit on 14 Nov — Rs. 4.2L (approximately 8.5× your average credit). This may be one-time income such as a bonus or asset sale, which affects your advance tax Q3 computation.”*

Academic justification

This introduces an unsupervised anomaly detection component into the pipeline — a second, independently evaluable ML module beyond the RAG system. The thesis can report precision and recall against a manually labelled set of anomalous transactions. It also demonstrates domain-specific prompt engineering: the contextualisation of a raw anomaly score into a tax-relevant explanation.

4.4. Extension 4: Fine-Tuned Domain Embeddings on ICAI Corpus

Limitation addressed: #7 (generic embeddings).

What changes

`all-MiniLM-L6-v2` (80 MB, fits entirely within 4 GB VRAM) is fine-tuned on a dataset of question–chunk pairs automatically generated from ICAI study PDFs using Gemini Flash. Training uses `MultipleNegativesRankingLoss` from `sentence-transformers`. The resulting checkpoint replaces the generic `bge-m3` model, and new `.joblib` embedding files are generated from the fine-tuned model.

Concrete example: Before fine-tuning: query “*penalty for late GST return*” retrieves a generic penalty chunk unrelated to GST. **After fine-tuning:** retrieves the exact GSTR-3B late fee provision (Rs. 50/day for non-nil returns, Rs. 20/day for nil returns) from the ICAI Indirect Tax module. RAGAS context recall improves from 0.71 to 0.89 (baseline established in Extension 5).

Academic justification

Domain adaptation of sentence embeddings is one of the most impactful contributions a final year project can make, because it is rigorous (requires a training pipeline, dataset curation, and quantitative evaluation), reproducible (the fine-tuned model and dataset can be published), and practically significant (improves every downstream retrieval query). The before/after RAGAS comparison is the core empirical finding of the thesis.

4.5. Extension 5: RAGAS Evaluation Harness

Limitation addressed: #5 (no retrieval quality measurement).

What changes

`metrics.py` is fully activated and integrated into a new `run_eval.py` script. A golden dataset of 100+ question–answer–chunk triples is curated from ICAI Foundation, Intermediate, and Final study material. The script reports RAGAS metrics: **faithfulness** (is the answer supported by retrieved chunks?), **answer relevancy** (does the answer address the question?), and **context recall** (were the relevant chunks retrieved?).

Concrete example: The golden dataset includes entries such as:

- Question: “*What is the threshold turnover for GST registration for a service provider?*”
- Ground truth: “*Rs. 20 lakh (Rs. 10 lakh for special category states).*”
- Expected source chunk: ICAI Indirect Tax, Chapter 2, Section 2.3.

Running the harness before and after Extension 4 produces the quantitative improvement table for the thesis.

Academic justification

The majority of student RAG projects have no quantitative evaluation at all. A RAGAS benchmark with a curated domain dataset is publishable-quality evidence. It also provides the before/after comparison that justifies every other retrieval improvement in the project. Without this, claims of improvement are anecdotal.

4.6. Extension 6: GraphRAG on Income Tax Act Cross-References

Limitation addressed: Structural gap in the retrieval model.

What changes

A directed graph is constructed from ICAI text using `networkx`: nodes are Act sections (e.g. Section 80C, Section 10(13A), Section 44AD), and edges represent cross-references found in the text (“as per Section X” or “subject to Section Y”). At query time, the top vector-retrieved section seeds a breadth-first traversal of depth 2. The union of vector results and graph neighbours is passed to the reranker.

Concrete example: Query: “*Can I claim both HRA exemption and home loan deduction?*” Vector retrieval alone returns Section 10(13A) (HRA exemption). GraphRAG additionally traverses to Section 24 (home loan interest deduction) and Section 80C (principal repayment deduction) via the cross-reference edges. The final answer correctly addresses all three sections and their interaction — the answer a CA would actually give.

Academic justification

GraphRAG (Edge et al., Microsoft, 2024) is one of the most cited recent advances in retrieval-augmented generation. Applying it specifically to a structured legal corpus with known cross-reference patterns is a meaningful domain-specific instantiation. The thesis can compare vector-only vs. hybrid GraphRAG on multi-hop questions from the golden dataset. Runs entirely on CPU; no additional GPU requirement.

4.7. Extension 7: Hallucination Detection and Source Badges

Limitation addressed: #6 (no reliability signal).

What changes

A Natural Language Inference (NLI) CrossEncoder (`cross-encoder/nli-MiniLM2-L6-H768`, approximately 90 MB) is applied post-generation. It scores each sentence of the generated answer against the top retrieved chunks. If the entailment score exceeds 0.70, the response is tagged **ICAI-verified** with the source citation. Otherwise it is tagged **Inferred** — **verify with your CA**. Both badges are rendered in `ChatMessage.tsx`.

Concrete example: Agent answers: “*Section 44AD allows presumptive taxation at 8% of gross turnover for eligible businesses.*” The NLI model finds direct textual entailment with the retrieved ICAI chunk → badge displayed: **ICAI-verified (Foundation Tax, Chapter 9)**. If the agent had hallucinated “6% of net turnover”, the NLI model would find contradiction → badge: **Inferred — verify with your CA**.

Academic justification

Hallucination in LLMs for high-stakes domains (legal, medical, financial) is a central concern in current NLP research (Huang et al., 2023). Adding an NLI-based grounding layer to a financial RAG system is a direct response to this concern and a novel contribution: the system not only retrieves and answers, it audits its own output. The thesis can measure badge accuracy against human-labelled responses.

4.8. Extension 8: Agentic ITR Filing Assistant

Limitation addressed: Moves the system from reactive Q&A to task completion.

What changes

A four-step agent is implemented in `ca_agent.py` as a state machine:

- Step 1. Classify** — determine ITR form (ITR-1, ITR-2, ITR-4) from income type and business status extracted from `doc_data`.
- Step 2. Gap-fill** — identify missing information required for the selected form and ask the user targeted clarifying questions.
- Step 3. Pre-fill** — populate all extractable fields from the uploaded documents and conversation.
- Step 4. Export** — generate a structured pre-filled summary (PDF or JSON) with a checklist of fields requiring manual CA input.

A new `POST /itr` endpoint and `ITRWizard.tsx` frontend component expose this as a guided wizard in the UI.

Concrete example: A salaried user uploads salary slips and a bank statement. **Step 1:** System classifies ITR-1 (income below Rs. 50L, single employer, no business income). **Step 2:** Asks for Form 16 and home loan certificate. **Step 3:** Pre-fills gross salary, TDS, employer PAN, and 80C investments from the available documents. **Step 4:** Exports a checklist: 3 fields auto-filled, 2 fields need manual entry, 1 document missing.

Academic justification

This moves the project from a question-answering system to an agentic task-completion system — a qualitative leap in system complexity and user value. The state machine architecture, gap-identification logic, and structured export constitute distinct engineering contributions. For a final year project, a working agentic workflow over real financial documents is significantly more impressive than a chatbot alone.

4.9. Extension 9: Cloud Deployment with In-Process Embeddings

Limitation addressed: #3 (Ollama dependency blocking cloud deployment).

What changes

The Ollama HTTP call in `search.py` is replaced with a direct `SentenceTransformer.encode()` call using the fine-tuned model from Extension 4. The backend is containerised in a `Dockerfile` (`python:3.11-slim` base, `.joblib` files copied in, Uvicorn as entrypoint). The frontend is deployed to Vercel (free tier); the

backend to Render.com or Railway.app (free tier). Environment variables are managed via the platform dashboards.

Concrete example: The deployed system is accessible at a public URL. The examiner can open it on any device, upload a sample bank statement, and receive an AI-generated financial summary and tax estimate — without any local setup. The entire system runs at zero cost using Gemini Flash’s free tier (1 M tokens/month) and Render/Vercel free plans.

Academic justification

A deployed, publicly accessible system is strong evidence of engineering competence and project completeness. It removes the “works on my machine” risk during the viva. The containerisation also satisfies the reproducibility requirement of academic projects: the Docker image encapsulates the exact environment, making the system independently runnable by any evaluator.

5. Why These Extensions Make a Strong Final Year Project

5.1. Multi-Dimensional Contribution

A common weakness of student AI projects is that they have a single contribution (e.g. “we built a chatbot”). CA Agent with the proposed extensions has contributions across four distinct dimensions, each evaluable independently:

Dimension	Contribution	Extensions
ML research	Fine-tuned domain embeddings; RAGAS evaluation; anomaly detection; NLI grounding	Ext. 3, 4, 5, 7
System design	GraphRAG hybrid retrieval; agentic state machine; rolling memory; multi-document fusion	Ext. 1, 2, 6, 8
Software engineering	Cloud deployment; Docker; in-process embeddings; dashboard analytics	Ext. 2, 9
Domain expertise	ICAI-aligned Q&A dataset; ITR classification logic; GST compliance rules; Indian tax law graph	Ext. 4, 5, 6, 8

5.2. Quantitative Evaluability

Strong final year projects are judged partly on whether their claims can be verified. The extensions provide three independently measurable results:

1. **RAGAS scores** (before/after fine-tuning, before/after GraphRAG): faithfulness, answer relevancy, context recall — each a real number that can be tabulated and plotted.
2. **Anomaly detection precision/recall** against a manually labelled transaction dataset.
3. **Hallucination badge accuracy**: percentage of ICAI-verified badges that correspond to genuinely grounded claims, validated against human annotation.

These three evaluation axes mean the thesis has a dedicated results chapter with tables and graphs, not just screenshots.

5.3. Relation to Current Literature

Each extension engages with a recognised research problem:

- **Domain-specific RAG** — Guu et al. (REALM, 2020); Lewis et al. (RAG, 2020); Shi et al. (domain adaptation, 2023).
- **GraphRAG** — Edge et al. (Microsoft, 2024); knowledge graph enhanced retrieval.
- **Hallucination detection** — Huang et al. (2023 survey); NLI-based factual consistency (Falke et al., 2019).
- **Conversational memory** — MemGPT (Packer et al., 2023); summarisation-based context management.
- **Agentic LLM systems** — ReAct (Yao et al., 2022); tool-augmented agents (Schick et al., ToolFormer, 2023).

The project is not merely applying existing tools; it is instantiating and evaluating these techniques in a specific high-value domain (Indian taxation) that has not been studied in the NLP literature.

5.4. Differentiation from Existing Tools

Capability	ClearTax	Quicko	TaxBuddy	CA Agent (proposed)
Conversational Q&A	No	No	Limited	Yes
Document vision extraction	No	No	No	Yes (Gemini Vision)
ICAI-grounded retrieval	No	No	No	Yes (RAG + GraphRAG)
Hallucination detection	—	—	—	Yes (NLI badges)
Multi-document analysis	No	No	No	Yes
Anomaly flagging	No	No	No	Yes
ITR agent workflow	Partial	Partial	No	Yes (agentic)
Compliance reminders	Yes	Yes	No	Yes
Open source / extensible	No	No	No	Yes

6. Hardware Feasibility

All proposed extensions are validated against the available hardware (i5-12th gen, 16 GB RAM, 4 GB VRAM):

VRAM and compute allocation

- **Fine-tuning** (Extension 4): `all-MiniLM-L6-v2` is 80 MB. With batch size 16 and mixed precision, training requires approximately 2.5 GB VRAM. Fits comfortably.

- **Inference** (all extensions): The CrossEncoder reranker and NLI model total under 200 MB. Both run well on CPU if VRAM is occupied during fine-tuning.
- **GraphRAG** (Extension 6): **networkx** graph operations are CPU-only. The tax law graph for the full Income Tax Act is estimated at under 50 MB in memory.
- **Anomaly detection** (Extension 3): **IsolationForest** on ≤ 500 transaction rows completes in under 100 ms on CPU.
- **LLM inference**: All generative tasks (answer generation, summarisation, HyDE) use Gemini Flash API. No local LLM required. Gemini Flash free tier: 1 million tokens/-month, sufficient for development and evaluation.

7. Proposed Evaluation Plan

What is evaluated	Method	Target metric
Retrieval quality (baseline)	RAGAS on 100-item golden dataset	Context recall > 0.75
Retrieval quality (fine-tuned)	RAGAS on same dataset after Extension 4	Context recall > 0.88
GraphRAG improvement	RAGAS on 30 multi-hop questions	Context recall > baseline
Anomaly detection	Precision/recall vs. 50 labelled statements	F1 > 0.80
Hallucination badge accuracy	Human annotation of 50 responses	Precision > 0.85
ITR classification accuracy	20 test cases with known correct form	Accuracy > 0.90
System latency (deployed)	Average response time over 100 queries	< 5 seconds

8. Summary

CA Agent is an existing, working prototype that already demonstrates a complete end-to-end AI pipeline: document vision extraction, domain-specific RAG, and a conversational interface. The nine proposed extensions convert it from a prototype into a research-grade system by:

1. Fixing known architectural limitations (memory, Ollama dependency, single-document isolation).
2. Adding domain-adapted ML components with quantitative before/after evaluation (fine-tuned embeddings, RAGAS harness).
3. Introducing novel retrieval techniques applied to a new domain (GraphRAG on Indian tax law cross-references).
4. Building trust mechanisms appropriate for a high-stakes domain (NLI-based hallucination detection with source badges).
5. Delivering a working agentic task-completion flow (ITR filing assistant) that goes beyond conversational Q&A.

-
6. Demonstrating full-stack engineering capability through containerised cloud deployment.

The result is a project with multiple independent contributions, three quantitative evaluation axes, engagement with current NLP literature, and a live publicly accessible demo — the combination that distinguishes a strong final year project from an average one.